

Sample Unit Plan + Complementary Lessons

ThriveUp Academy — Software Engineering Enrichment (Grades 6–12)

Submitted in support of AISD 26RFP052 Proposal Response Form Question 10

About this sample

This unit is drawn from our production 15-lesson Software Engineering track — one of six trade tracks (electrical, plumbing, HVAC, welding, automotive, software engineering) in the ThriveUp Academy Trade-Skills curriculum. The full track contains 15 sequenced lessons with formative practice banks, an AI tutor in Socratic mode for guided hints, and a 6-certification ladder unlocked at 80% lesson completion. For this RFP we are sharing **Unit 1 of the Software Engineering track (3 lessons of 15)** as a representative example of our unit-planning rigor.

Core teaching stance: "Anybody can vibe code. You have to know how the system works to make vibecoding work."^{*} This is the deliberate pedagogical position of the entire track — AI-assisted code generation is a real tool, but durable competence is in systems thinking. Every lesson reinforces this stance.

UNIT 1 OVERVIEW

Field	Value
Unit Title	"How the Computer Hears You" — Foundations of Computational Thinking
Grade Band	6–12 (differentiated within-band)
Duration	3 × 1-hour lessons (1 unit week) · expandable to 6 × 1-hour with optional extension
Setting	Afterschool, weekend, intersession, or summer enrichment block
Group Size	Up to 15 (1:15 ratio); 1:10 if hands-on lab
Materials Provided by ThriveUp	Laptops with offline-capable lessons (provided as needed if campus lacks 1:1), printed reference packs, formative-assessment cards, instructor guide
Materials Required from AISD	Programming space with seating, internet (preferred but not required), one power outlet per 4 students

TEKS Alignment Crosswalk (Unit 1)

TEKS Standard	Alignment to this unit
§126.32 (Fundamentals of Computer Science)	Knowledge & Skills (c)(2)(B–C): demonstrate ethical use; (c)(4)(A): solve problems individually and collaboratively using a systematic approach
§126.33 (Computer Science I)	(c)(1)(C): identify the major components of a computer system; (c)(2)(A): use a systematic problem-solving process
§126.7 (Technology Applications, Grade 6)	(c)(1)(A): create original products using a variety of resources; (c)(4): critical thinking, problem solving, decision making
ELAR §110 (grade-band)	Speaking & listening: explain reasoning, build on others' ideas during structured discourse

Critical Success Factor Targets (per RFP Attachment B)

- Improved school-day attendance (mechanism: positive reinforcement in Math and Science process strands (mechanism: systematic problem-solving process))
- Increased secondary aspiration (mechanism: visible evidence of achievement)
- Improved behavior (on-campus pathway)

collaborative, success-experience-rich activities)

Unit Essential Question

"When you type something into an AI chatbot or a search engine, what actually happens between your fingers and the answer that comes back?"

Unit Objectives

By the end of this 3-lesson unit, learners will be able to:

1. Describe in their own words the four foundational layers of a computing system (hardware !' operating system !' application !' user interface) between a user and a computer.
2. Apply the Search and Pattern Matching !' Abstract !' Algorithm* problem-solving sequence to a real, learner-chosen everyday task (making a sandwich, getting to school, etc.)
3. Apply the Search and Pattern Matching !' Abstract !' Algorithm* problem-solving sequence to a real, learner-chosen everyday task (making a sandwich, getting to school, etc.) without systems understanding produces brittle results, citing one concrete example from class discussion.

Unit Assessment

- Formative: Exit-ticket prompts at each lesson (open-ended).
- Summative: A learner-chosen "Algorithm of a Daily Task" produced individually or in pairs, presented in 90 seconds to the class.
- No grade is posted to the campus; results inform our program improvement only and a personalized "next steps" recommendation for each learner who wants one.

LESSON 1 — "Inside the Box"

Target Concepts: layered system architecture

TEKS: 8-126.33(c)(4)(C), 8-126.32(c)(4)(A)

Materials

- One student per group: 1. A large sheet of paper (divided) labeled photograph

Lesson Flow

Opening (3-5 minutes)

The teacher asks two questions, in this order: *"Raise your hand if you've ever talked to an AI chatbot. Now keep your hand up if you know what happens between your typing and its answer."* (Most hands up)

Forming (5-10 minutes) Today we're going to figure out what's actually in the box. Not so you can build one tomorrow — so you know what you're actually using when you click a button.*

Direct Instruction (12 minutes)

The teacher shows a 4-layer model using a simple analogy (a building: foundation !' plumbing/wiring !' rooms !' furniture). Each layer is named, defined in one sentence.

Hardware — the physical project (CPU, memory, storage, etc.)

Operating system — the traffic controller that lets everything else use the hardware (Windows, macOS, Linux, etc.)

Application — what you actually opened (the browser, a game, etc.)

User interface — the part of the application you see and touch

Guided Practice (15 minutes) The teacher shows the system-diagram handout and asks students to label one common scenario at all four layers: *

"What is the hardware, the OS, the application, and the user interface when you ask ChatGPT a question on a school Chromebook?"* (Answer: Chromebook hardware, ChromeOS, web browser, chat input box.)

Discussion (12 minutes) (whole group) Plants and harvests this question: *"Could you 'vibe code' a

Conclusion (10 minutes) Why software lives on top of hardware. You can vibe at the top layer; the bottom layers have to be real.

Closing (5 minutes) "Name one thing you use everyday and one of its four layers" your phone" video; learners label additional layers.

Differentiation

- **Grade 6–7:** Use only the building analogy throughout; the system diagram is pre-labeled with two of Grade 11–12. Add this new layer ("the wire to the outside world") and ask learners to label where the AI's actual answer-generation happens (it's not on the CPU).
- **ELL learners:** It's translated layer labels are available in 89 languages; pair work intentionally pairs an ELL learner with a strong English speaker who shares a home-language tile.

LESSON 2 — "The Right Question"

Target Concept: Computational decomposition — breaking a problem into smaller, simpler parts.

Materials

- Sticky notes (60 per group) with the example step (provided)

Lesson Flow

Opening (5 minutes) 30 seconds, soliciting one volunteer. Instructor reads an intentionally bad algorithm aloud: "To make breakfast: 1. Wake up. 2. Make breakfast."* Wait for laughter. "Why doesn't this work?"*
Direct instruction (10 minutes) Tasks have small steps inside them. The computer can't do the big task — focus on only the central steps.* as a single phrase the class repeats: ***Decompose ! Pattern-Match !***
Abstract the Algorithm The recipe is a decomposed task. The recipe author already did the systems thinking. **Model of a decomposed task** suggesting to school in the morning" on the board with the class adding the steps. **Abstract the Algorithm** The class takes 12 steps and collapses them into 2 steps ("eat, brush teeth, leave") into a stage that could be re-used for "getting to grandma's house."*
Group work (15 minutes) Each pair must identify one step in the other pair's algorithm that's actually two steps mashed together.
Discussion (10 minutes) Provide the breakdown: "Where did you have the step? Why would it break apart?"* answer and the answer is bad, almost always it's because the question wasn't decomposed enough.*

Closing (5 minutes)

Review Lesson 3 — learners will turn their decomposition into a 90-second presentation.

Differentiation

- **Grade 6–7:** Provide a pre-decomposed scaffold (3 steps).
- **Grade 11–12:** A learner must explicitly write down what could go wrong at each step, in their own words.
- **Neurodiverse learners:** Use a visual decomposition with icons/drawings instead of words for any learner who prefers.

LESSON 3 — "Why Systems Beat Vibes"

Target Concept: Algorithm presentation; systems thinking as the differentiator between durable and brittle tasks. ELAR 126.32(c)(4)(A); ELAR speaking-and-listening standards (grade-banded)
Materials

- **Pre-recorded Coding Demo:** prepared by the instructor (a live AI prompt that produces a plausible but slightly wrong answer to a real task — e.g., a brittle pasta-cooking instruction set).

Lesson Flow

Opening (5 minutes) with the "Vibe Coding Demo" — types a sloppy prompt into an AI chatbot in front of the class and asks the AI for a recipe with no decomposition. The AI produces a plausible answer that contains a real error (e.g., "boil pasta for 30 minutes").
Question to the class: "What did the AI do wrong? Did the AI know it was wrong?"
Direct instruction (8 minutes) point: AI generates plausible-looking outputs from patterns. Without the human supplying the systems thinking (the decomposition, the constraints, the edge cases), the AI's output is brittle.
Example prompt: named: "Anybody can vibe code."

You have to know how the system works to make vibecoding work! not anti-AI. This is what makes you the engineer rather than the chatbot's stenographer.

Presentations (or 30 minutes) its their algorithm-of-a-day in 90 seconds.
Activity in 90 seconds: What's one step you'd refine if you had a structural engineering firm on the board. The instructor plans a 40-minute segment on the board. Group presentations (10 minutes) did you learn this well that you could defect to work the systems-thinking thread — every learner today did the work of an engineer.

Closing (2 minutes) is a 20-second prompt: "Abstract! Why is systems thinking important when you use AI?"
Pathway preview: For learners who want more, the next 12 lessons take them deeper — including registered-apprenticeship and entry-level Software Engineering certifications. Optional sign-up for the full track.

Differentiation

- **Grade 6–7:** Presentations can be 60 seconds; learners may present in their home language with translated subtitles.
- **Grade 11–12:** Presenters required to identify one real-world consequence if their algorithm were applied at scale (e.g., "if my route-to-school algorithm were used by a school bus company, the consequence of forgetting Step 4 would be...").

ASSESSMENT RUBRIC — Unit 1 Summative

The 90-second algorithm presentation in Lesson 3 is the summative assessment. Rubric (4-point scale per dimension, internal only, not reported as a grade to AISD):

Dimension	4 = Strong	3 = Developing	2 = Emerging	1 = Not yet
Decomposition	All major steps identified; no compound steps	Most major steps identified	Many steps are still compound	Task not meaningfully broken down
Sequence	Steps in a workable order	Mostly workable order; one obvious gap	Order has multiple errors	Sequence not communicated

Abstraction	Identifies reusable groupings	Some grouping attempted	Grouping not yet attempted	N/A
Systems explanation	Articulates <i>*why*</i> the order matters	Names the rule but partial reasoning	Order asserted without reasoning	Cannot articulate yet

Internal use only; learner gets a coach-style narrative back, not a numeric score.

FAMILY ENGAGEMENT CONNECTION (per National PTA Standard #3)

A 1-page take-home is provided in 89 languages at the end of Lesson 3 inviting families to attend a free Family Tech Night where their learner shares their algorithm-of-a-daily-task with them — and the family member is invited to add their own task. This satisfies **National PTA Standard #3: Active parent participation in student learning.**

INSTRUCTOR PREPARATION

- 90-minute pre-unit instructor briefing (recorded; available on-demand for ~~substitutes~~ **walkthrough** of the Lesson 3 "Vibe Coding Demo" — instructor practices the prompt and the AI's likely answer twice before ~~delivering~~ **delivering** assembled and shipped 5 business days before the first ~~lesson~~ **LESSON** crosswalk shared with campus Roadmap point-of-

contact for alignment
confirmation.

*This sample is one unit (3 of 15 lessons) within the Software Engineering track. The complete 15-lesson sequence covers: Foundations of Computational Thinking · The Internet · How Code Works · Version Control Basics · Debugging as a Skill · Working in Teams · APIs and Why They Matter · Data and How to Think About It · Building a First App · Testing Your Work · Deployment and the World Beyond · Ethics in Software · Career Pathways · Apprenticeship Routes · Certification
Prep *
Prepared for AISD 26RFP052 Proposal Response · The Collaborative Advocate Foundation / ThriveUp Academy · 2026-05-18